Tarea 2: Árboles B y B^ε en memoria externa

CC4102 Diseño y Análisis de Algoritmos · Semestre 2026-2

Julio Yáñez · 01-11-2026

1. Introducción

En memoria externa, el costo de una operación se mide en transferencias de bloques entre el disco y la memoria principal (I/Os). El árbol B es la estructura clásica para un diccionario ordenado en este modelo: cada nodo ocupa un bloque con $\Theta(B)$ llaves, por lo que insertar y buscar cuesta $\Theta(\log_B N)$ I/Os. El árbol B^ε cambia ese equilibrio a favor de las inserciones: cada nodo interno usa solo del orden de B^ε llaves divisorias y destina el resto del bloque a un *buffer* de inserciones pendientes, que bajan a los hijos en lotes cuando el buffer se llena. Así, cada I/O de un vaciado mueve muchos pares a la vez y una inserción cuesta $O(\log_B N/(\varepsilon B^{1-\varepsilon}))$ I/Os amortizadas, a cambio de una altura $O(\log_B N/\varepsilon)$, que es lo que pagan las búsquedas.

En esta tarea se implementaron en C++ un árbol B ($b = 340$) y dos árboles B^ε, todos con nodos de 4096 bytes: uno con $\varepsilon \approx 0{,}25$ ($c = 4$, $l_{\max} = 506$) y otro con $\varepsilon \approx 0{,}75$ ($c = 126$, $l_{\max} = 384$). Con ellos se indexaron los primeros $N \in \{2^{15}, ..., 2^{26}\}$ pares (tiempo Unix, temperatura) medidos en la estación Quinta Normal entre 2019 y 2025. Los árboles se construyen sobre un arreglo de bloques en RAM que simula el disco y cuenta cada lectura y escritura; después se guardan en un archivo binario y se hacen 50 búsquedas de rangos de una semana leyendo los nodos desde ese archivo. Se reportan el tiempo y las I/Os de la construcción, el tamaño y la altura de cada árbol, y el tiempo y las I/Os de las búsquedas. En el experimento bonus se estudia además el parámetro α del vaciado con $N = 2^{24}$.

La Sección 2 describe las estructuras, los algoritmos y las decisiones de implementación que el enunciado deja abiertas. La Sección 3 presenta el entorno, la metodología y los resultados; la Sección 4 los analiza con el análisis amortizado de los vaciados y la altura en función de ε, y la Sección 5 concluye.

Hipótesis. Un bloque de 4096 bytes tiene espacio para $B = 512$ pares de 8 bytes (el nodo del árbol B guarda 340, porque también almacena los índices de los hijos), y $B^{0{,}25} \approx 4{,}8$ y $B^{0{,}75} \approx 108$, que corresponden aproximadamente a los 5 y 127 hijos de las dos configuraciones.

1. *Inserción.* El árbol B lee el camino desde la raíz y escribe la hoja: unas $h + 1$ I/Os por inserción, con una altura h de 2 a 4 para estos N. En un árbol B^ε cada inserción lee y escribe la raíz (2 I/Os) y los vaciados agregan poco, porque cada lote mueve muchos pares: con $\varepsilon \approx 0{,}25$ el hijo más poblado recibe al menos $\lceil 506/5 \rceil = 102$ pares por vaciado, pero con $\varepsilon \approx 0{,}75$ solo $\lceil 384/127 \rceil = 4$. Se espera entonces, en I/Os por inserción, B^ε (0,25) $<$ B^ε (0,75) $<$ B, con el primero muy cerca de 2. Como en la simulación cada I/O copia un bloque de 4 KiB, el tiempo de creación debería seguir el mismo orden.
2. *Altura y búsqueda.* La altura de un B^ε crece como $1/\varepsilon$: el árbol con $\varepsilon \approx 0{,}25$ debería tener unas 3 veces la altura del de 0,75, que a su vez debería ser casi tan bajo como el árbol B. En búsquedas que tocan pocos nodos eso ordena los árboles como B $\approx$ B^ε (0,75) $<$ B^ε (0,25). Pero un rango de una semana contiene $\approx 0{,}29$ % de los datos (casi 200 000 pares con $N = 2^{26}$, cientos de bloques), así que para N grande debería dominar el número de bloques con resultados y los tres árboles deberían acercarse.
3. *Tamaño.* Las hojas de los B^ε tienen espacio para más pares que las del árbol B (506 y 384 contra 340), así que deberían ocupar menos bloques, en especial con $c = 4$.

2. Desarrollo

2.1. Organización del código

Los árboles y los experimentos están en C++17 (`src/`). El código de los gráficos está aparte, en Python (`graficos/graficar.py`), y solo lee los CSV que produce el programa en `resultados/`. Los árboles son plantillas parametrizadas por b o por $(c, l_{\max})$: el programa usa $b = 340$, $(4, 506)$ y $(126, 384)$, y las pruebas usan además valores chicos como $b = 4$ o $(c, l_{\max}) = (4, 6)$, que generan árboles profundos con

pocos datos y ejercitan todos los casos de split y vaciado. Cada estructura y función está documentada en el código con su propósito y sus parámetros de entrada y salida.

Archivo	Contenido
`par.hpp`, `arbol_comun.hpp`	par llave-valor de 8 bytes (mismo formato que `datos.bin`), inserción ordenada y mezcla de lotes
`disco.hpp`	disco simulado en RAM: vector de bloques con conteo de lecturas/escrituras y serialización a archivo
`archivo.hpp`	escritura de archivos de bloques y lectura de un nodo con `pread` y `O_DIRECT`
`arbol_b.hpp`, `arbol_be.hpp`	árbol B y árbol B^ε: splits, vaciado, inserción y búsqueda por rango
`datos.hpp`, `main.cpp`	lectura por bloques de `datos.bin`; experimento principal y bonus de α
`tests/pruebas.cpp`	pruebas de correctitud (invariantes, I/Os exactas y comparación con fuerza bruta)

Tabla 1: Archivos del programa.

2.2. Nodos, disco simulado y archivo

Los nodos son `struct` con exactamente los campos del enunciado y en el mismo orden (`NodoB<340>`, `NodoBe<4,506>` y `NodoBe<126,384>`); un `static_assert` comprueba que cada uno mide 4096 bytes. Como no contienen punteros, un nodo se guarda y se lee copiando sus bytes.

El disco simulado es un `std::vector<Nodo>` en RAM con tres operaciones, y los algoritmos solo acceden a los nodos a través de ellas: `leer(i, nodo)` copia el bloque i completo a una variable local y cuenta una lectura; `escribir(i, nodo)` copia la variable al bloque i y cuenta una escritura; y `agregar(nodo)` la inserta al final del arreglo, cuenta una escritura y retorna su posición. Como los algoritmos trabajan siempre sobre copias locales, igual que si hubieran traído el bloque del disco, los contadores son exactamente la cantidad de bloques transferidos. La raíz está siempre en el bloque 0.

Al terminar las inserciones, el árbol se escribe en un archivo binario, bloque por bloque y en orden (el nodo i queda en el byte $i \cdot 4096$), se hace `fsync` y se le pide al sistema operativo que saque el archivo de su caché de páginas (`posix_fadvise`). Después se destruye el árbol en RAM. Las búsquedas abren el archivo con `O_DIRECT`: leer el nodo i es un `pread` de 4096 bytes en la posición $i \cdot 4096$ hacia un buffer alineado, que va al dispositivo sin pasar por la caché del sistema operativo. Sin `O_DIRECT`, después de la primera búsqueda el archivo quedaría en esa caché y se mediría la velocidad de la RAM. Cada nodo leído es una I/O de búsqueda.

2.3. Árbol B

La inserción sigue el esquema del enunciado, con división preventiva: antes de bajar a un hijo lleno ($k = b$), se lo divide en el par de la posición $b/2 = 170$, que deja 169 pares a la izquierda y 170 a la derecha. U_{der} se agrega al final, U_{izq} reemplaza a U y el padre, que siempre tiene espacio porque nunca se entra a un nodo lleno, recibe la mediana y se escribe. La inserción continúa en la mitad que corresponde, que ya está en memoria y no se vuelve a leer. Una raíz llena se divide igual: ambas mitades van al final del arreglo y la nueva raíz al bloque 0.

La búsqueda por rango recorre cada nodo en orden (hijo, par, hijo, …) y baja solo a los hijos cuyo rango intersecta $[l, u]$. Con a el primer índice cuya llave es $\geq l$ y z el primero cuya llave es $> u$ (búsquedas binarias), visita los hijos $a, ..., z$ y reporta los pares $a, ..., z-1$. Por eso el resultado queda ordenado por llave.

2.4. Árbol B^ε

Inserción. Cada inserción lee la raíz y la escribe con el par agregado a su buffer. Si la raíz es una hoja llena, se divide en la posición $l_{\max}/2$, como indica el enunciado, y se crea una raíz interna con el par en su buffer. Si la raíz es interna y su buffer está lleno, primero se vacía; si con eso llega a c llaves, se divide y se crea una raíz nueva en el bloque 0.

Vaciado de un nodo V. Como el buffer está ordenado, el grupo de cada hijo es un tramo contiguo, cuyos límites se encuentran con una búsqueda binaria por llave divisoria. Con $f = \lceil k_{\text{buffer}}/(k_{\text{llaves}} + 1) \rceil$ se eligen los hijos que tienen al menos $\lceil \alpha f \rceil$ pares; los pares de los demás se quedan en V. Los hijos elegidos se procesan de derecha a izquierda: para cada uno se lee el hijo U y se le entrega su lote de m pares.

- Si U es interno y $k_{\text{buffer}}(U) + m > l_{\max}$, se vacía U (recursivamente) hasta que el lote quepa. Después se mezcla el lote con el buffer de U (mezcla de dos arreglos ordenados).

- Si U es una hoja y el lote no cabe, se divide la hoja (ver abajo).

Durante su vaciado, V se mantiene en memoria como una *copia de trabajo* y se escribe una sola vez, al final, por quien lo llamó. Así, el vaciado no agrega I/Os propias (el nodo ya está en memoria) y cada lote cuesta 1 lectura y 1 escritura del hijo, más 1 escritura por cada nodo nuevo que aparezca al dividirlo (Tabla 2).

Decisiones propias en los splits.

- *Split de hoja al recibir un lote.* El enunciado divide la hoja en la posición $l_{\max}/2$ y después reparte el lote, pero si el lote cae entero a un lado esa mitad puede quedar con más de $l_{\max}$ pares. En vez de eso, se mezclan los $s = k_{\text{buffer}} + m$ pares ($s \le 2l_{\max}$) y se cortan en la mediana: los primeros $\lceil s/2 \rceil$ quedan en U_{izq}, en la posición de U, y el resto en U_{der}, al final del arreglo. La llave del último par de U_{izq} sube como divisoria. Es lo mismo que dividir y repartir, pero garantiza que ambas hojas quedan con entre $l_{\max}/2$ y $l_{\max}$ pares.
- *Split de nodo interno.* Si en un mismo vaciado se dividen varios hijos, V recibe varias llaves nuevas. Como V está en memoria, se aceptan todas y, antes de escribirlo, se divide en su llave mediana (la de índice $\lfloor (K-1)/2 \rfloor$ de sus K llaves, desde 0), repitiendo en cada mitad que siga con más de c llaves; el buffer se reparte en $\le k_m$ y $> k_m$. Con $K = c + 1$ quedan $c/2$ llaves en cada mitad (2 y 2 con $c = 4$; 63 y 63 con $c = 126$). Con $K = c$, el caso habitual de la raíz (que se divide en cuanto tiene c o más llaves), resultan las $c/2 - 1$ y $c/2$ llaves del enunciado. Las mitades nuevas se agregan al final y sus llaves se insertan en el padre justo a la derecha del hijo dividido; como los hijos se procesan de derecha a izquierda, los índices de los que faltan no cambian.

Búsqueda. En cada nodo se reportan los pares del buffer que están en $[l, u]$ (dos búsquedas binarias) y, si es interno, se baja a los hijos cuyo rango intersecta $[l, u]$, como en el árbol B. El resultado no queda ordenado, porque el buffer de un nodo puede tener pares posteriores a los de sus hijos; se ordena solo al escribirlo en un CSV.

Llaves repetidas. Los datos tienen 11 llaves repetidas. Un split puede dejar pares con la misma llave a ambos lados de una llave divisoria, así que en ambos árboles se visita un hijo si su rango *cerrado* $[k_{i-1}, k_i]$ intersecta $[l, u]$.

Operación	Lecturas	Escrituras
Árbol B: insertar un par sin splits (altura h)	h	1
Árbol B: split de un nodo o de la raíz	0	3
B^ε: insertar un par en el buffer de la raíz	1	1
B^ε: entregar un lote a un hijo	1	1
B^ε: cada nodo nuevo que crea un split	0	1
Búsqueda (ambos): cada nodo visitado, desde el archivo	1	0

Tabla 2: I/Os que cuenta la implementación por cada operación.

2.5. Experimentos y verificación

El ejecutable `build/tarea2` dispone de los comandos `principal` ($N = 2^{15}, ..., 2^{26}$ con los tres árboles) y `alpha` (experimento bonus). Para cada N y cada árbol:

1. se leen los primeros N pares de `datos.bin` en bloques de 65 536 y se insertan uno a uno; el tiempo de creación (`steady_clock`) cuenta solo las inserciones, no la lectura del archivo;
2. se registran las I/Os simuladas y la estructura del árbol (nodos, altura, ocupación, vaciados, lotes y splits);
3. se guarda el árbol en disco y se libera su RAM;
4. se hacen las búsquedas sobre el archivo, midiendo el tiempo y las I/Os de cada una.

Los rangos se generan una vez con `mt19937_64` y semilla fija: l uniforme en $[1\,546\,300\,800, 1\,754\,006\,400]$ y $u = l + 604\,800$. Son los mismos para todos los N y árboles, lo que permite comparar una misma búsqueda. Además, `tests/pruebas.cpp` realiza 30 678 verificaciones (0 fallas): cuenta las I/Os exactas de casos pequeños hechos a mano; valida todas las invariantes después de cada inserción en árboles chicos (orden, rangos de los subárboles, ocupación, hojas al mismo nivel, altura, ningún nodo perdido y el mismo multiconjunto de pares que se insertó), y compara búsquedas con fuerza bruta sobre datos aleatorios, ordenados, con llaves repetidas y sobre una muestra de `datos.bin`.

3. Resultados

3.1. Entorno

Componente	Valor
Sistema operativo	Ubuntu / Linux kernel 7.0.0-38-generic (x86_64)
Procesador	AMD Ryzen 7 7840HS (8 núcleos, 16 hilos)
Cachés	L1d: 32 KiB × 8 (256 KiB); L1i: 32 KiB × 8 (256 KiB); L2: 1 MiB × 8 (8 MiB); L3: 16 MiB
RAM	14 GiB DDR5 (swap de 511 MiB)
Compilador	`g++ -std=c++17 -O2 -Wall -Wextra -Wpedantic`

Tabla 3: Entorno de hardware y software en que se ejecutaron los experimentos.

3.2. Datos y metodología

`datos.bin` (512 MiB) contiene $2^{26} = 67\,108\,864$ pares en orden aleatorio, con llaves entre $1\,546\,300\,802$ (01-01-2019 00:00:02 UTC) y $1\,754\,006\,394$ (31-07-2025 23:59:54 UTC), 11 llaves repetidas y temperaturas entre $-2,53$ y $38,32$ °C. Para cada N se usan los primeros N pares; como el archivo está desordenado, cada prefijo es una muestra aleatoria de todo el período y está contenido en los prefijos más grandes.

Experimento principal. Para cada $N \in \{2^{15}, ..., 2^{26}\}$ se construyeron los tres árboles (36 construcciones) y cada construcción se siguió de las mismas 50 búsquedas (1 800 búsquedas en total). Ambos árboles B^ε utilizan $\alpha = 0{,}9$ (justificado en la Sección 3.7). Todas las búsquedas leyeron directo del disco (`O_DIRECT`).

Bonus. Con $N = 2^{24}$, para el árbol B^ε con $c = 126$ ($\varepsilon \approx 0{,}75$) y cada uno de los 8 valores de α $(0{,}1; 0{,}25; 0{,}4; 0{,}5; 0{,}6; 0{,}75; 0{,}9; 0{,}99)$ se construyó el árbol y se ejecutaron 20 búsquedas en rangos aleatorios de una semana.

3.3. Creación

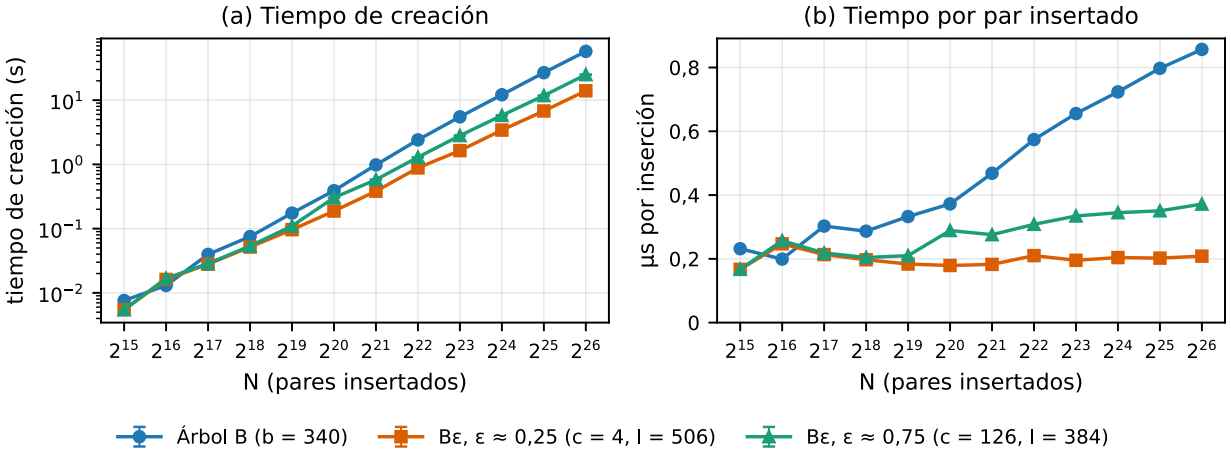


Figura 1: Tiempo de creación (inserción de los N pares) en función de N: (a) total; (b) por par insertado.

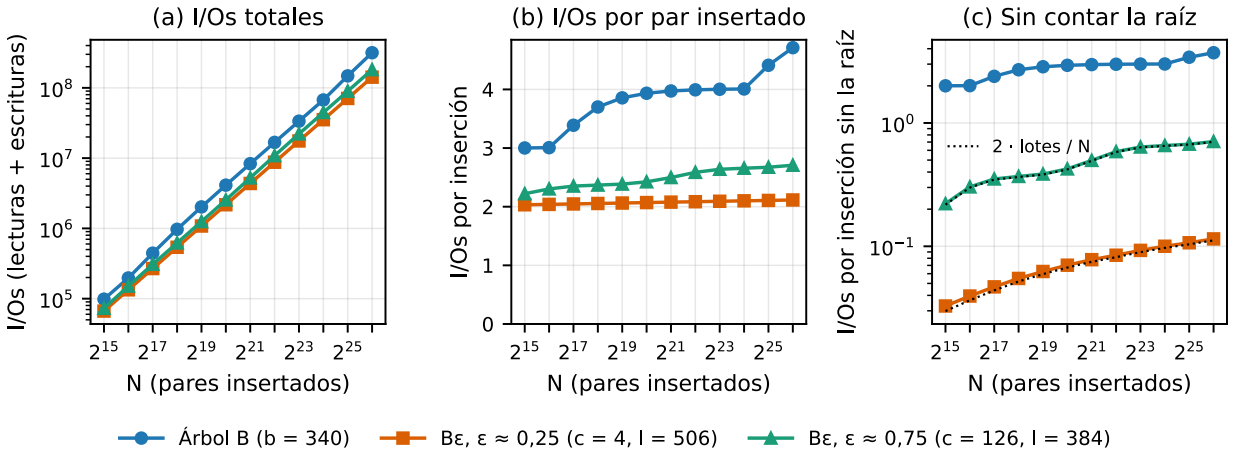


Figura 2: I/Os simuladas al insertar los N pares: (a) totales; (b) por par insertado; (c) por par sin contar la raíz (1 lectura en el árbol B, 1 lectura y 1 escritura en los B^ε). La línea punteada es $2 \cdot \text{lotes}\,/N$: leer y escribir el hijo en cada lote de un vaciado.

- El tiempo de creación crece casi linealmente con N (Figura 1 a). El orden B^ε $c = 4 <$ B^ε $c = 126 <$ B se cumple en 11 de los 12 valores de N. Con $N = 2^{26}$: 14,0 s, 24,9 s y 57,5 s.
- Por par insertado (Figura 1 b), entre $N = 2^{15}$ y $N = 2^{26}$ el árbol B pasa de 0,24 a 0,86 µs, el B^ε $c =$ 126 de 0,16 a 0,37 µs y el B^ε $c = 4$ de 0,16 a 0,21 µs.
- Las I/Os por inserción (Figura 2 b) pasan de 3,002 a 4,712 en el árbol B, de 2,221 a 2,707 en el B^ε $c = 126$ y de 2,033 a 2,115 en el B^ε $c = 4$. El orden B^ε $c = 4 <$ B^ε $c = 126 <$ B se cumple para todo N.
- Sin contar la raíz (Figura 2 c), con $N = 2^{26}$ el árbol B hace 3,71 I/Os por inserción, el B^ε $c = 126$ 0,707 y el B^ε $c = 4$ 0,115, es decir, 5,3 y 32 veces menos que el árbol B. En ambos B^ε, la línea $2 \cdot$ lotes $/N$ queda a menos de 1,7 % ($c = 126$) y 9,0 % ($c = 4$) de la curva medida.

Medida	**B**	**B^ε c=4**	**B^ε c=126**
Nodos internos	1 041	64 893	3 094
Hojas	278 430	139 200	250 234
% de los pares en nodos internos	0,41	29,87	1,14
Pares por hoja (promedio)	240,0	338,1	265,1
Vaciados	-	1 953 618	753 963
Pares por lote (promedio)	-	158,0	8,4
Bajadas por par	-	8,83	2,95
Lecturas por par insertado	3,700	1,056	1,352
Escrituras por par insertado	1,012	1,059	1,355
ms por I/O de búsqueda	0,055	0,058	0,060

Tabla 4: Estructura de los árboles y contadores de la inserción con $N = 2^{26}$. *Bajadas por par*: total de pares bajados un nivel (sumando todos los lotes) dividido por N. La última fila es el tiempo de búsqueda dividido por sus I/Os.

3.4. Tamaño y altura

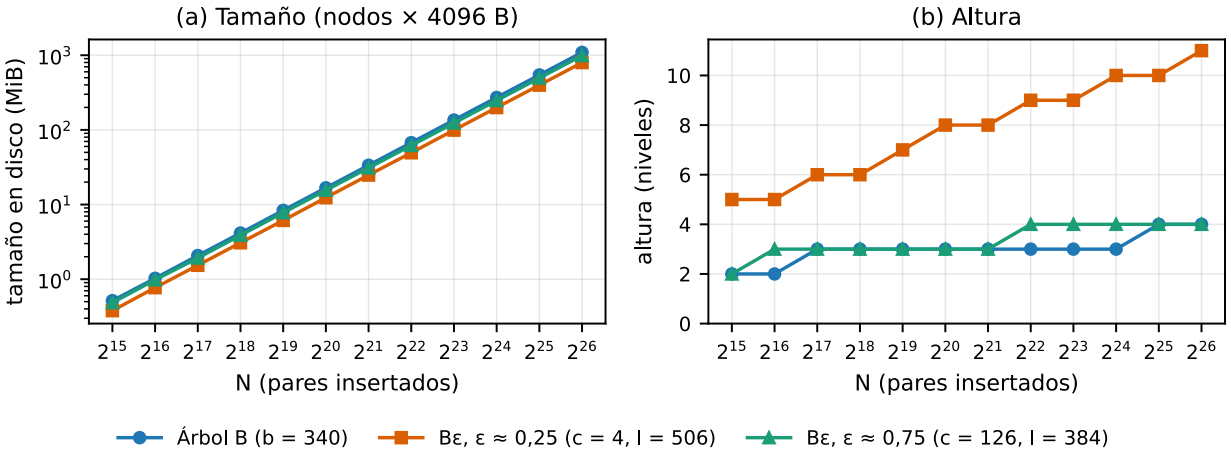


Figura 3: (a) Tamaño de los árboles (cantidad de nodos por 4096 bytes, que es el tamaño del archivo) y (b) altura, en función de N.

- El tamaño crece proporcionalmente a N (Figura 3 a). Con $N = 2^{26}$, el árbol B usa 279 471 nodos (1 091,7 MiB); el B^ε $c = 126$, 253 328 (989,6 MiB, 9 % menos), y el B^ε $c = 4$, 204 093 (797,2 MiB, 27 % menos). El orden es el mismo para todo N.
- La altura (Figura 3 b) del árbol B pasa de 2 a 4 (sube en 2^{17} y 2^{25}) y la del B^ε $c = 126$ de 2 a 4 (sube en 2^{16} y 2^{22}). La del B^ε $c = 4$ pasa de 5 a 11 (sube en $2^{17}, 2^{19}, 2^{20}, 2^{22}, 2^{24}$ y 2^{26}).
- Las hojas del árbol B tienen en promedio 240 pares (71 % de 340); las del B^ε $c = 126$, 265 (69 % de 384), y las del B^ε $c = 4$, 338 (67 % de 506). Además, el 29,9 % de los pares del B^ε $c = 4$ está en buffers de nodos internos, contra 1,1 % en el de $c = 126$ y 0,4 % en los nodos internos del árbol B (Tabla 4).

3.5. Búsquedas

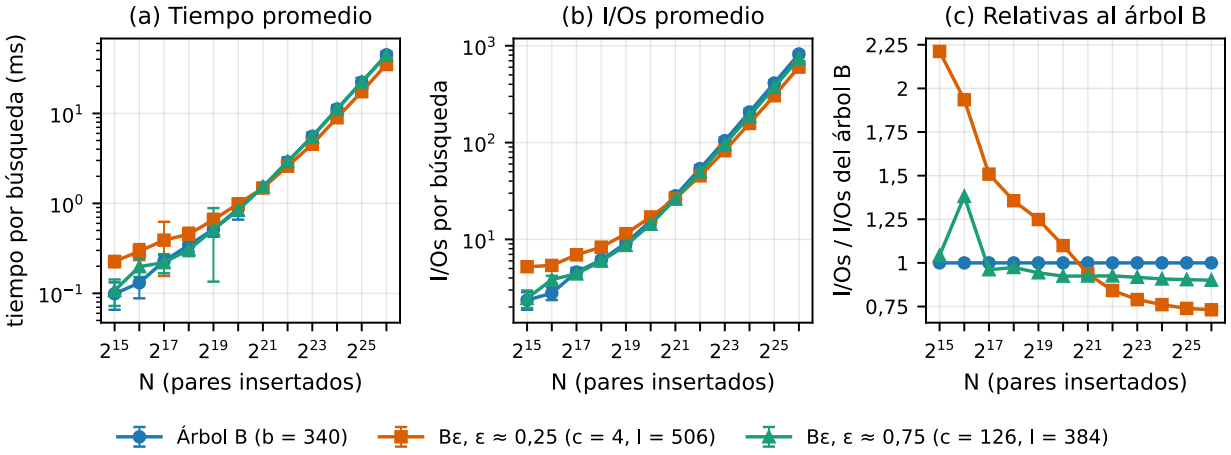


Figura 4: Búsquedas de rangos de una semana (promedio de las 50 búsquedas) en función de N: (a) tiempo por búsqueda y (b) I/Os por búsqueda, con barras de ± una desviación estándar; (c) I/Os de cada árbol divididas por las del árbol B.

- Cada búsqueda devolvió en promedio el 0,291 % de los N pares (lo esperado para rangos de una semana es 0,291 %), desde 97 pares con $N = 2^{15}$ hasta 195 420 con $N = 2^{26}$; los tres árboles devolvieron exactamente los mismos pares.
- Las I/Os por búsqueda (Figura 4 b) crecen casi proporcionalmente a N cuando N es grande. Con $N = 2^{15}$ el B^ε $c = 4$ lee 5,2 bloques, contra 2,4 del árbol B (2,21 veces) y 2,5 del B^ε $c = 126$. Desde $N = 2^{22}$, el B^ε $c = 4$ es el que menos lee (Figura 4 c): con $N = 2^{26}$ hace 601,5 I/Os, contra 740,7 del B^ε $c = 126$ y 822,4 del árbol B (0,73 y 0,90 veces las del árbol B).

- El tiempo de búsqueda (Figura 4 a) sigue a las I/Os: con $N = 2^{26}$, 35,0 ms (B^ε $c = 4$), 44,2 ms (B^ε $c = 126$) y 44,9 ms (B), entre 0,05 y 0,06 ms por I/O (Tabla 4). Así, el B^ε $c = 4$ es más lento que los otros dos con $N \leq 2^{20}$ y el más rápido desde $N = 2^{22}$.

3.6. Una misma búsqueda con dos valores de N

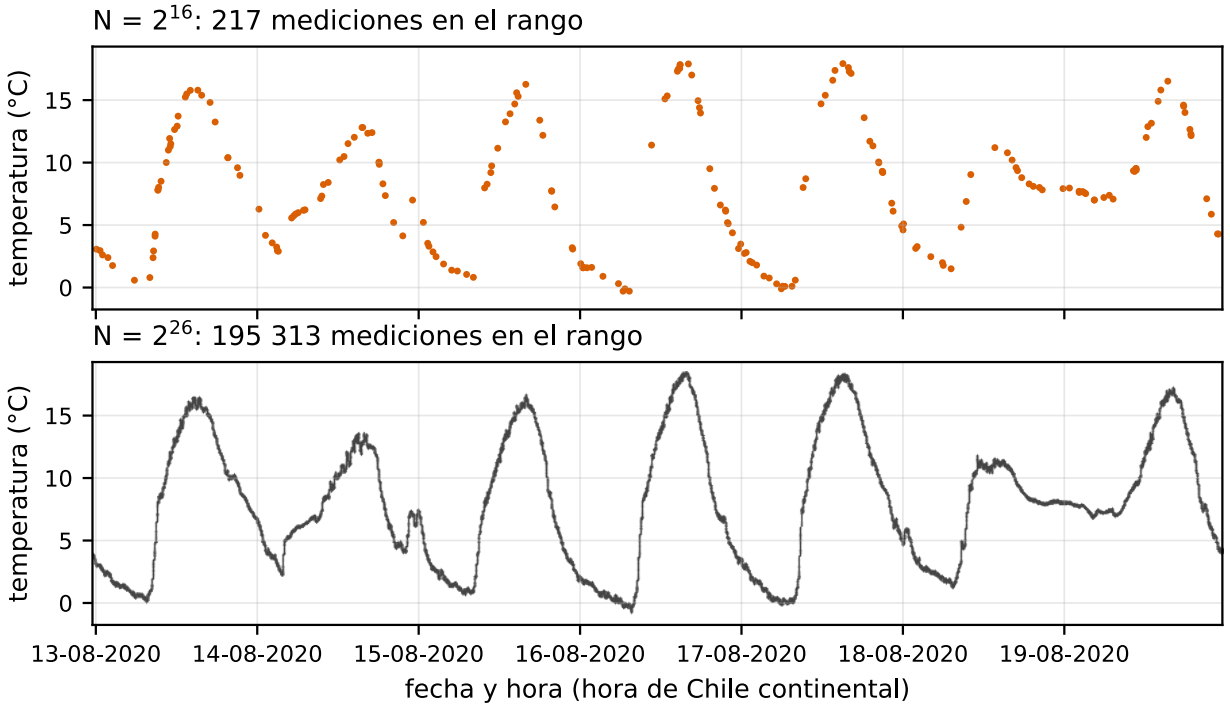


Figura 5: Pares que devuelve la búsqueda 0, [1 597 289 407, 1 597 894 207] (del 12-08-2020 a las 23:30 al 19-08-2020 a las 23:30, hora de Chile), con $N = 2^{16}$ y 2^{26}.

N	Pares	% de N	Mín. (°C)	Máx. (°C)	Prom. (°C)	Separación mediana (s)	¿Contenido en el de 2^{26}?
2^{16}	217	0,331	-0,29	17,92	8,21	2 214	sí
2^{26}	195 313	0,291	-0,80	18,33	7,86	2	sí

Tabla 5: Resumen de la búsqueda 0 para $N = 2^{16}$ y 2^{26}. *Separación mediana*: mediana del tiempo entre dos mediciones consecutivas del resultado.

La búsqueda 0 cubre una semana (Figura 5). Con $N = 2^{16}$ devuelve 217 mediciones, separadas por 37 minutos en la mediana, y con $N = 2^{26}$ devuelve 195 313, separadas por 2,0 segundos (Tabla 5). Las dos muestran el mismo ciclo diario: en los 6 días completos del rango, el máximo diario va de 11,7 a 18,3 °C y el mínimo diario de $-0,8$ a 2,2 °C (con $N = 2^{26}$). El promedio de la semana cambia poco entre ambos valores de N (8,2 y 7,9 °C). Todos los pares del resultado con $N = 2^{16}$ aparecen en el de $N = 2^{26}$, porque los prefijos están contenidos unos en otros. Con $N = 2^{16}$ se pierden las variaciones que duran menos que la separación entre mediciones, que con $N = 2^{26}$ se ven como una curva continua.

3.7. Bonus: elección de α

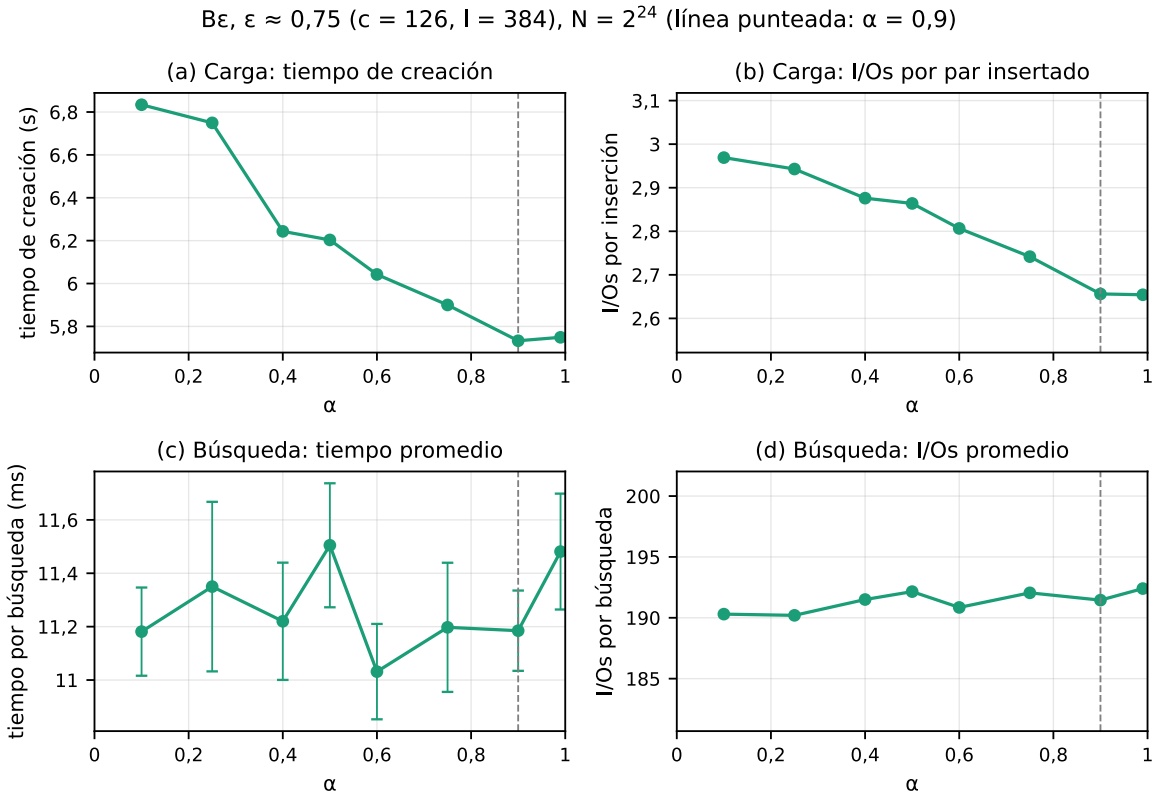


Figura 6: Experimento de α con el B^ε $c = 126$ y $N = 2^{24}$. (a) Tiempo de creación; (b) I/Os por inserción; (c) tiempo por búsqueda (promedio de 20 búsquedas ± un error estándar); (d) I/Os por búsqueda. Las escalas de (b) y (d) van de 5 % bajo el mínimo a 5 % sobre el máximo.

α	t creación (s)	desv. (s)	I/Os por par	Vaciados	Pares por lote	% pares en internos	Nodos	t búsq. (ms)	err. est. (ms)	I/Os búsq.
0,1	6,83	0,00	2,969	130 648	5,9	0,91	63 374	11,2	0,2	190,3
0,25	6,75	0,00	2,943	131 095	6,0	0,85	63 402	11,4	0,3	190,2
0,4	6,24	0,00	2,876	133 630	6,5	0,91	63 393	11,2	0,2	191,5
0,5	6,20	0,00	2,864	134 741	6,6	0,91	63 405	11,5	0,2	192,2
0,6	6,04	0,00	2,807	142 611	7,0	0,99	63 292	11,0	0,2	190,8
0,75	5,90	0,00	2,742	154 777	7,7	1,01	63 466	11,2	0,2	192,1
0,9	5,73	0,00	2,656	186 200	8,6	1,15	63 361	11,2	0,2	191,4
0,99	5,75	0,00	2,654	187 999	8,7	1,13	63 336	11,5	0,2	192,4

Tabla 6: Experimento de α con el B^ε $c = 126$ y $N = 2^{24}$.

En el B^ε con $c = 126$ (Figura 6, Tabla 6), las I/Os de carga bajan de forma monótona desde 2,969 por inserción con $\alpha = 0{,}1$ hasta 2,654 con $\alpha = 0{,}99$, una reducción de 10,6 % (32 % sin contar la raíz). El lote promedio pasa de 5,9 a 8,7 pares, los vaciados de 130 648 a 187 999 y la fracción de pares en nodos internos de 0,9 % a 1,1 %. El tiempo de creación baja de 6,83 s a 5,73 s. En cambio, las I/Os de búsqueda quedan entre 190,2 y 192,4 (solo 1,2 % de variación) y el tiempo de búsqueda entre 11,0 y 11,5 ms.

Elección. Se escogió $\alpha = 0{,}9$: sus I/Os de carga (2,656 por par) están a solo 0,07 % del mínimo (2,654 con $\alpha = 0{,}99$), tuvo el menor tiempo de creación (5,73 s) y sus I/Os de búsqueda están a 0,7 % del mínimo, evitando forzar el caso extremo $\alpha \to 1$ que incrementa el número de vaciados. En este experimento una mejor carga no empeoró la búsqueda, porque las I/Os de búsqueda varían apenas un 1,2 % a lo largo de todo el rango de α.

4. Análisis

4.1. Costo amortizado de la inserción

Árbol B. Cada inserción lee los h nodos del camino y escribe la hoja. Un split escribe 3 bloques y ocurre en promedio una vez cada ≈ 240 inserciones en una hoja (cada split crea una hoja nueva, y las hojas tienen en promedio 240 pares), lo que suma unas $3/240$ escrituras por inserción: con $N = 2^{26}$ el árbol B hace 1,012 escrituras por par (exactamente $1 + N + 3 \cdot \text{splits}$). Por lo tanto, las I/Os por inserción son $h + 1$ más esa pequeña fracción, lo que explica los escalones de la Figura 2 b: suben un poco más de 1 cada vez que el árbol gana un nivel (en 2^{17} y 2^{25}).

Árbol B^ε. Cada inserción cuesta 2 I/Os en la raíz. El resto son lotes: cada uno cuesta 2 I/Os (leer y escribir el hijo), más una escritura por nodo nuevo (las lecturas son exactamente $N + \text{lotes}$ y las escrituras $N + \text{lotes} + \text{nodos}$). Si $\overline{m}$ es el tamaño promedio de un lote y $\overline{d}$ el número promedio de niveles que baja cada par,

$$\text{I/Os por inserción} \approx 2 + \frac{2\overline{d}}{\overline{m}}. \tag{1}$$

Con los contadores de la Tabla 4, la Ecuación 1 da $2 + 2 \cdot 8{,}83 \;/\; 158{,}0 = 2{,}112$ para $c = 4$ y $2 + 2 \cdot 2{,}95 \;/\; 8{,}4 = 2{,}703$ para $c = 126$, a 0,003 y 0,004 de lo medido (2,115 y 2,707). Esa diferencia son las escrituras de los nodos nuevos, que la ecuación no incluye.

El análisis amortizado acota ambos factores. Un par baja a lo más $h - 1 = O(\log_{B^\varepsilon} N) = O(\log_B N/\varepsilon)$ niveles. Y cada lote mueve muchos pares si el buffer está casi lleno al vaciarse: con $\approx l_{\max}$ pares repartidos entre a lo más $c + 1 \approx B^\varepsilon$ hijos, el hijo más poblado recibe al menos $f \approx l_{\max}/(c+1) = \Omega(B^{1-\varepsilon})$ pares, y cada hijo elegido, al menos αf. Con un argumento de potencial, en el que cada par en un buffer guarda crédito para sus bajadas futuras y cada bajada cuesta $O(1/(\alpha B^{1-\varepsilon}))$ I/Os, el costo amortizado de los vaciados es

$$O\left(\frac{h}{\alpha B^{1-\varepsilon}}\right) = O\left(\frac{\log_B N}{\alpha\varepsilon B^{1-\varepsilon}}\right) \text{ I/Os por inserción.} \tag{2}$$

En los datos, con $\varepsilon \approx 0{,}25$ los lotes son grandes ($\overline{m} = 158{,}0$, del orden de $506/\ 3{,}15 \approx 161$, porque los nodos internos tienen en promedio 3,15 hijos): aunque cada par baja 8,83 niveles, cada bajada cuesta solo 0,01 I/Os. Con $\varepsilon \approx 0{,}75$ los lotes son de 8,4 pares: cada par baja solo 2,95 niveles, pero cada bajada cuesta 0,24 I/Os. La teoría predice que el costo del árbol B dividido por el de un B^ε (sin la raíz) es $\Theta(\varepsilon B^{1-\varepsilon})$: ≈ 27 para $\varepsilon = 0{,}25$ y $\approx 3{,}6$ para $\varepsilon = 0{,}75$. Lo medido con $N = 2^{26}$ es 32 y 5,3.

La raíz domina. En el modelo de la tarea la raíz se lee y se escribe en cada inserción, y esas 2 I/Os son el 95 % de las I/Os del B^ε $c = 4$ y el 74 % de las del B^ε $c = 126$. Por eso, contando todo, la ventaja de los B^ε sobre el árbol B es de 2,2 y 1,7 veces. En un sistema real la raíz estaría siempre en RAM, y la diferencia efectiva es la de la Figura 2 c.

Tiempo. En la simulación, el tiempo de creación sigue a las I/Os por el costo de copiar un bloque de 4 KiB: 0,06–0,09 µs por I/O con $N \le 2^{20}$, cuando los árboles (≤ 17 MiB) caben en las cachés, y 0,10–0,16 µs con $N = 2^{26}$, cuando ocupan 0,78–1,1 GiB. El B^ε $c = 4$ es el más barato por I/O (0,10 µs contra 0,16 µs del árbol B) porque casi todas sus I/Os acceden a la raíz, que permanece en caché L1/L2.

4.2. Altura en función de ε y costo de búsqueda

Altura. En un árbol con H hojas cuyos nodos internos tienen en promedio F hijos, $h \approx 1 + \log_F H$. Con $N = 2^{26}$, F es 268 en el árbol B, 82 en el B^ε $c = 126$ y 3,15 en el B^ε $c = 4$. Eso da $\log_F H =$ 2,2, 2,8 y 10,3, acordes con los 3, 3 y 10 niveles internos medidos (alturas 4, 4 y 11). La razón entre los niveles internos de los dos B^ε ($10/3 \approx 3,3$) concuerda con la razón 3 entre sus $1/\varepsilon$, que predice $h = O(\log_B N/\varepsilon)$.

Búsqueda. Un rango que cubre una fracción p del espacio de llaves intersecta $\approx p \cdot n_\ell + 1$ de los n_ℓ nodos de cada nivel ℓ, así que

$$\text{I/Os por búsqueda} \approx p \cdot n + h, \tag{3}$$

donde n es el total de nodos. Con $p = 604\,800/207\,705\,600 = 0{,}29$ %, la Ecuación 3 queda a menos de 1,7 % de lo medido para $N \ge 2^{18}$ (Tabla 7). Esto explica la Figura 4 c:

- Con N chico, $p \cdot n < 1$ (a lo más 0,38 con $N = 2^{15}$) y manda la altura: con $N = 2^{15}$, el B^ε $c = 4$ ($h = 5$) hace 2,21 veces las I/Os del árbol B ($h = 2$).
- Con N grande manda $p \cdot n$ (con $N = 2^{26}$, de 594 a 814, contra alturas de 4 a 11), y el orden lo da el tamaño. El B^ε $c = 4$ tiene 27 % menos nodos que el árbol B porque sus hojas guardan 338 pares en promedio contra 240, y porque guarda el 29,9 % de los pares en buffers de nodos internos. Las curvas se cruzan en $N = 2^{22}$, cuando la menor cantidad de bloques compensa la mayor altura.

N	Árbol	Nodos n	Altura h	$p \cdot n + h$	I/Os medidas	Diferencia (%)
2^{20}	B	4 287	3	15,5	15,5	0,2
2^{20}	B^ε c=4	3 164	8	17,2	17,1	-0,9
2^{20}	B^ε c=126	3 979	3	14,6	14,3	-1,7
2^{23}	B	34 675	3	104,0	104,6	0,6
2^{23}	B^ε c=4	25 425	9	83,0	82,6	-0,5
2^{23}	B^ε c=126	31 650	4	96,2	95,9	-0,2
2^{26}	B	279 471	4	817,8	822,4	0,6
2^{26}	B^ε c=4	204 093	11	605,3	601,5	-0,6
2^{26}	B^ε c=126	253 328	4	741,6	740,7	-0,1

Tabla 7: I/Os por búsqueda: modelo de la Ecuación 3 y valor medido (promedio de las 50 búsquedas).

4.3. Efecto de α

Un lote cuesta 2 I/Os sin importar cuántos pares lleve, así que bajar grupos chicos desperdicia I/Os: si se dejan en el buffer, bajan más tarde junto con los pares que lleguen después. El umbral $\lceil \alpha f \rceil$ decide cuáles grupos bajan. Un α alto hace más vaciados, pero un vaciado no cuesta I/Os propias (el nodo ya se leyó), solo CPU para agrupar el buffer. Con $c = 126$, $f \approx 384/82 \approx 5$: con $\alpha = 0{,}1$ el umbral es 1 y baja todo grupo no vacío; con $\alpha = 0{,}9$ el umbral es 5 y los grupos pequeños esperan en el buffer, reduciendo en 32 % las I/Os fuera de la raíz. En la búsqueda, según la Ecuación 3 lo que importa para rangos de una semana es el número total de nodos, que con α varía solo un 0,3 %, por lo que el costo de búsqueda permanece prácticamente constante.

5. Conclusión

Se implementaron un árbol B y dos árboles B^ε con nodos de 4096 bytes. Se construyeron sobre un disco simulado que cuenta cada transferencia de bloque, se guardaron en archivos binarios y se ejecutaron búsquedas por rango leyendo desde disco para $N = 2^{15}, ..., 2^{26}$ pares reales de temperatura.

Hipótesis.

1. *Inserción: se cumplió.* En I/Os el orden B^ε $c = 4 < B^\varepsilon$ $c = 126 <$ B se cumplió para todo N, y en tiempo en 11 de 12 valores de N: con $N = 2^{26}$, 2,115, 2,707 y 4,712 I/Os por inserción, y 14,0, 24,9 y 57,5 s. Sin las 2 I/Os de la raíz, la ventaja sobre el árbol B es de 32 y 5,3 veces, explicada por el tamaño promedio de los lotes (158,0 pares con $\varepsilon \approx 0{,}25$ contra 8,4 con 0,75).
2. *Altura y búsqueda: se cumplió en parte.* Las alturas fueron las esperadas (11 contra 4 y 4 con $N = 2^{26}$), y con N chico el B^ε $c = 4$ fue el más costoso en búsqueda (2,21 veces las I/Os del árbol B con 2^{15}). Pero desde $N = 2^{22}$ fue el que menos leyó (0,73 veces las I/Os del árbol B en 2^{26}), ya que en rangos que abarcan cientos de bloques el costo es $p \cdot n + h$ y domina la compacidad del árbol.
3. *Tamaño: se cumplió.* Con $N = 2^{26}$, el B^ε $c = 4$ usó 27 % menos bloques que el árbol B y el de $c = 126$, 9 % menos.

Limitaciones y mejoras futuras.

- El modelo cuenta 2 I/Os de raíz por inserción, lo que oculta gran parte de la ventaja de los B^ε mantener la raíz y los primeros niveles en un *buffer pool* en RAM reflejaría mejor un sistema real.
- Solo se evaluaron rangos de una semana; variar el ancho del rango desde consultas puntuales hasta varios meses permitiría observar la transición entre el régimen dominado por la altura h y el dominado por el tamaño $p \cdot n$.
- No se implementó el borrado (no se pedía) ni la carga masiva (*bulk loading*) de datos preordenados.

Anexo: resultados completos del experimento principal

N	Árbol	t creación (s)	I/Os creación (millones)	I/Os por par	Nodos	MiB	Altura	t búsqueda (ms)	I/Os búsqueda	Resultados
2^{15}	B	0,008	0,10	3,002	132	0,5	2	0,10	2,4	97
	B^ε c=4	0,005	0,07	2,033	97	0,4	5	0,23	5,2	97
	B^ε c=126	0,005	0,07	2,221	124	0,5	2	0,11	2,5	97
2^{16}	B	0,013	0,20	3,007	263	1,0	2	0,13	2,8	191
	B^ε c=4	0,016	0,13	2,040	197	0,8	5	0,29	5,4	191
	B^ε c=126	0,017	0,15	2,303	251	1,0	3	0,20	3,8	191
2^{17}	B	0,040	0,44	3,389	531	2,1	3	0,23	4,6	380
	B^ε c=4	0,028	0,27	2,047	393	1,5	6	0,39	6,9	380
	B^ε c=126	0,029	0,31	2,351	492	1,9	3	0,22	4,4	380
2^{18}	B	0,075	0,97	3,700	1 062	4,1	3	0,34	6,1	761
	B^ε c=4	0,052	0,54	2,055	787	3,1	6	0,45	8,3	761
	B^ε c=126	0,054	0,62	2,368	992	3,9	3	0,31	6,0	761
2^{19}	B	0,174	2,02	3,856	2 139	8,4	3	0,51	9,2	1 525
	B^ε c=4	0,096	1,08	2,062	1 569	6,1	7	0,66	11,5	1 525
	B^ε c=126	0,110	1,25	2,385	1 991	7,8	3	0,51	8,7	1 525
2^{20}	B	0,390	4,13	3,934	4 287	16,7	3	0,87	15,5	3 049
	B^ε c=4	0,188	2,17	2,070	3 164	12,4	8	0,99	17,1	3 049
	B^ε c=126	0,303	2,54	2,424	3 979	15,5	3	0,84	14,3	3 049
2^{21}	B	0,982	8,33	3,973	8 601	33,6	3	1,51	28,2	6 086
	B^ε c=4	0,383	4,36	2,078	6 351	24,8	8	1,48	26,4	6 086
	B^ε c=126	0,578	5,24	2,497	7 930	31,0	3	1,53	26,1	6 086
2^{22}	B	2,408	16,75	3,993	17 266	67,4	3	2,90	53,9	12 207
	B^ε c=4	0,881	8,74	2,085	12 637	49,4	9	2,59	45,4	12 207
	B^ε c=126	1,293	10,84	2,586	15 843	61,9	4	2,92	49,9	12 207
2^{23}	B	5,500	33,58	4,003	34 675	135,4	3	5,57	104,6	24 448
	B^ε c=4	1,639	17,55	2,093	25 425	99,3	9	4,54	82,6	24 448
	B^ε c=126	2,804	22,13	2,638	31 650	123,6	4	5,51	95,9	24 448
2^{24}	B	12,140	67,24	4,008	69 581	271,8	3	11,20	207,1	48 865
	B^ε c=4	3,422	35,23	2,100	50 907	198,9	10	8,95	157,5	48 865
	B^ε c=126	5,782	44,56	2,656	63 361	247,5	4	11,14	188,0	48 865
2^{25}	B	26,757	148,03	4,412	139 394	544,5	4	22,27	411,8	97 706
	B^ε c=4	6,786	70,69	2,107	101 764	397,5	10	17,45	304,3	97 706
	B^ε c=126	11,764	89,68	2,673	126 696	494,9	4	22,34	372,2	97 706
2^{26}	B	57,493	316,22	4,712	279 471	1 091,7	4	44,85	822,4	195 420
	B^ε c=4	13,990	141,92	2,115	204 093	797,2	11	34,99	601,5	195 420
	B^ε c=126	24,939	181,67	2,707	253 328	989,6	4	44,23	740,7	195 420

Tabla 8: Resultados del experimento principal para $N = 2^{15}, ..., 2^{26}$.